

LAB REPORT: INTRODUCTION TO MODBUS (LAB 06)

Course: Intro to Critical Infrastructure for IT Learners

Students Name: WENDELL TWUMASI

Student ID: C11/26/ACIS/17355

Institution: International Cybersecurity and Digital Forensics Academy (ICDFA)

Submission Date: October 1, 2026

Primary Deliverables: Written Lab Report (<RegistrationNo>_LAB06_Introduction_to_Modbus.pdf) and Wireshark Capture (<RegistrationNo>_LAB06.pcapng)

1. Laboratory Topology & Environment Overview

The laboratory environment simulates an isolated Industrial Control System (ICS) / Operational Technology (OT) process network divided into L1 Process and L2 Control zones routed via a dual-homed gateway.

Entity / System	Virtual Machine	IP Address	Segment / Role	Function & Software Credentials
PLC Simulation	OpenPLC Runtime	192.168.1.10	L1 Process (192.168.1.0/24)	Modbus TCP Server (Port 502) Account: plc / password

Entity / System	Virtual Machine	IP Address	Segment / Role	Function & Software Credentials
SCADA / HMI	FUXA Web HMI	192.168.2.10	L2 Control (192.168.2.0/24)	Modbus TCP Client (Port 1881 Editor) Account: scada / password
Analyst Workstation	Kali Linux	192.168.2.5	L2 Control (192.168.2.0/24)	Packet Sniffing & Traffic Analysis Account: kali / kali
Gateway / Router	pfSense	192.168.1.1 / 192.168.2.1	Dual-Homed Inter-VLAN	Traffic Isolation & Network Routing

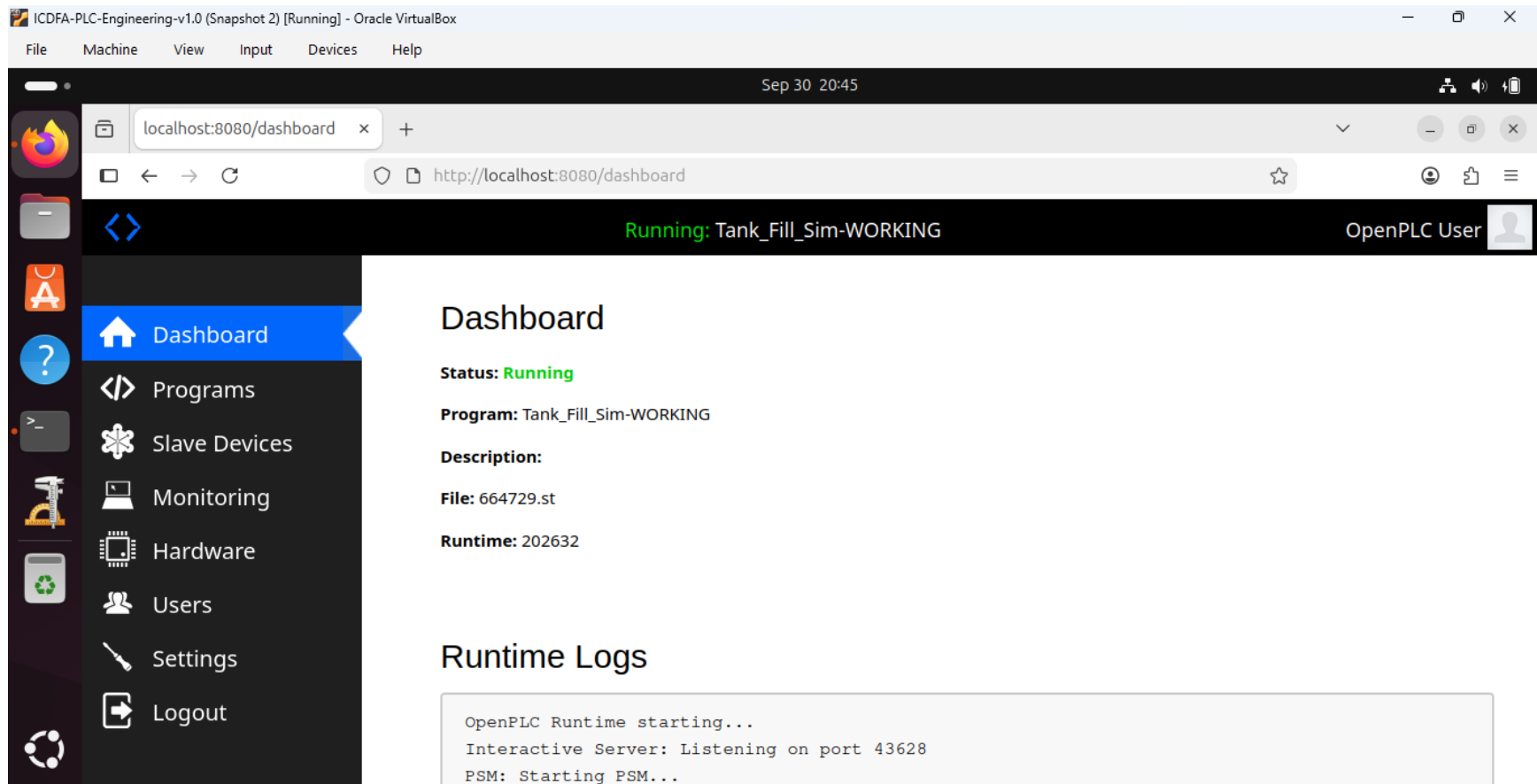
2. Practical Execution Walkthrough & Evidence Collection

Phase 1: OpenPLC Runtime Configuration & Execution

1. Logged into the PLC VM (192.168.1.10), opened Firefox, and accessed the OpenPLC Runtime web console at <http://localhost:8080/>.
2. Compiled and executed the discrete control program Tank_Fill_Sim-WORKING.st.
3. Confirmed on the dashboard that the program status indicated **Running** and that the server was actively listening on TCP Port 502.

SCREENSHOT Figure 1

Label: *Figure 1: OpenPLC Runtime Dashboard — Tank_Fill_Sim-WORKING Running*

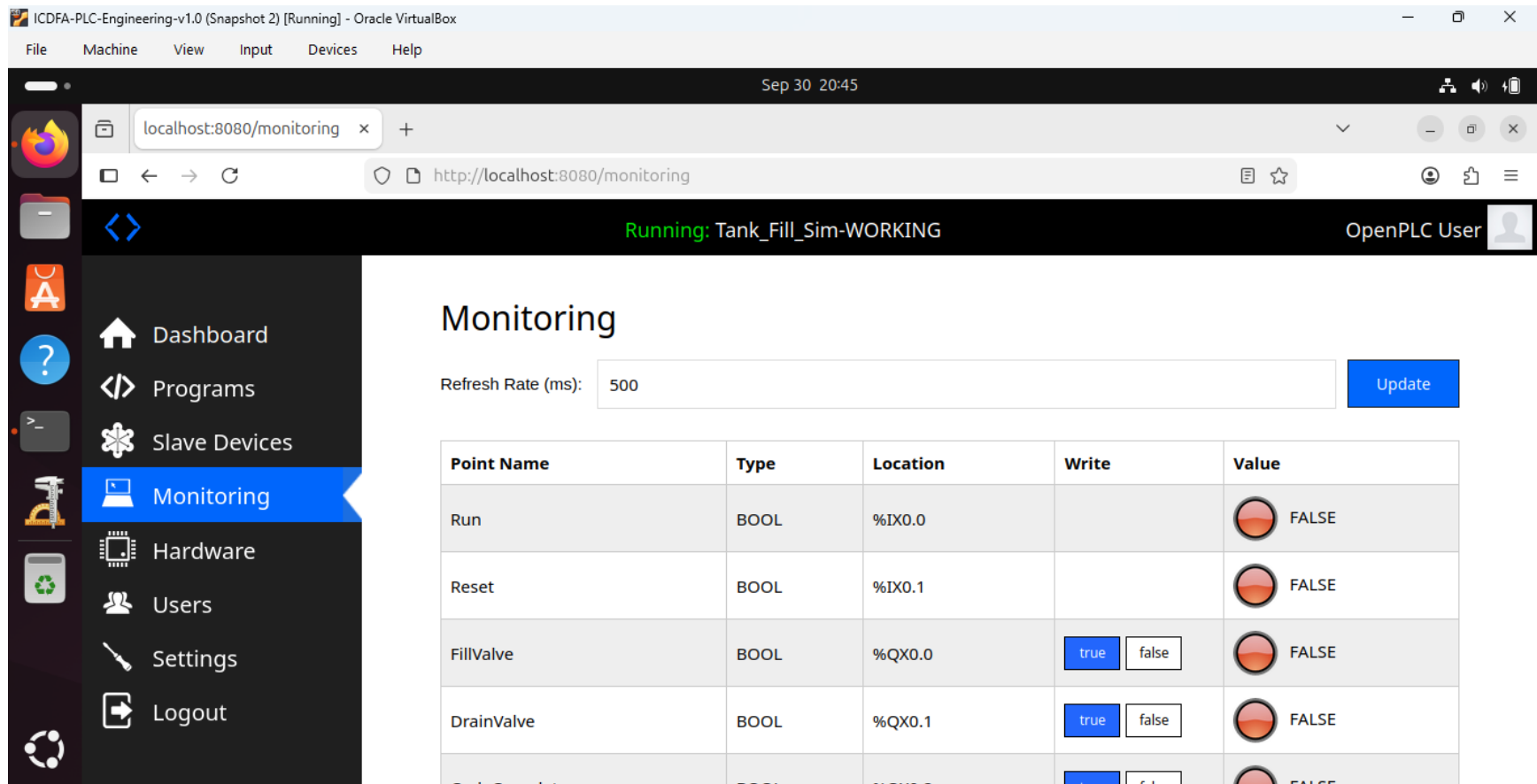


Technical Caption: Displays the active execution of Tank_Fill_Sim-WORKING.st on the OpenPLC server (192.168.1.10:502) with log output confirming the Modbus TCP socket initialization.

4. Navigated to the **Monitoring** tab (<http://localhost:8080/monitoring>) to verify IEC 61131-3 memory address assignments.

SCREENSHOT HERE: Figure 2

Label: *Figure 2: OpenPLC Monitoring Tab Variable Mappings*



Technical Caption: Verifies internal memory bindings mapping %IX0.0 (Run), %QX0.0 (FillValve), and %QX0.3 (HighLevel) to discrete Modbus address spaces.

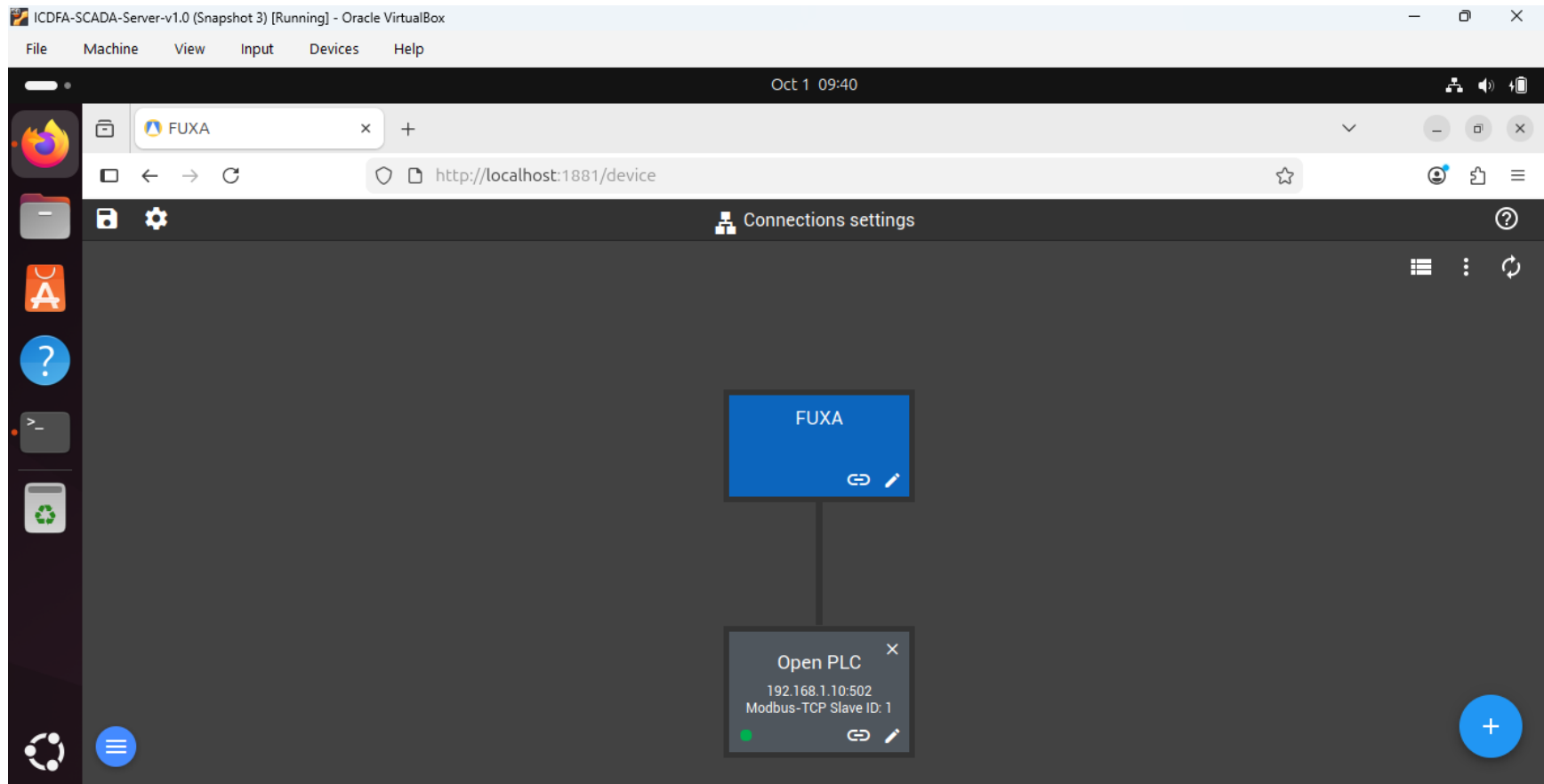
Phase 2: FUXA SCADA/HMI Configuration & Process Operation

1. Logged into the SCADA VM (192.168.2.10), launched FUXA via terminal (sudo fuxa), and accessed http://127.0.0.1:1881/editor.

2. Opened the project TankFillTest.json from Documents/FUXA Labs/.
3. Configured the Modbus TCP device connection (openplc) to target Slave IP 192.168.1.10, Port 502, Unit ID 1.

SCREENSHOT Figure 3

Label: *Figure 3: FUXA SCADA Modbus TCP Connection Settings*

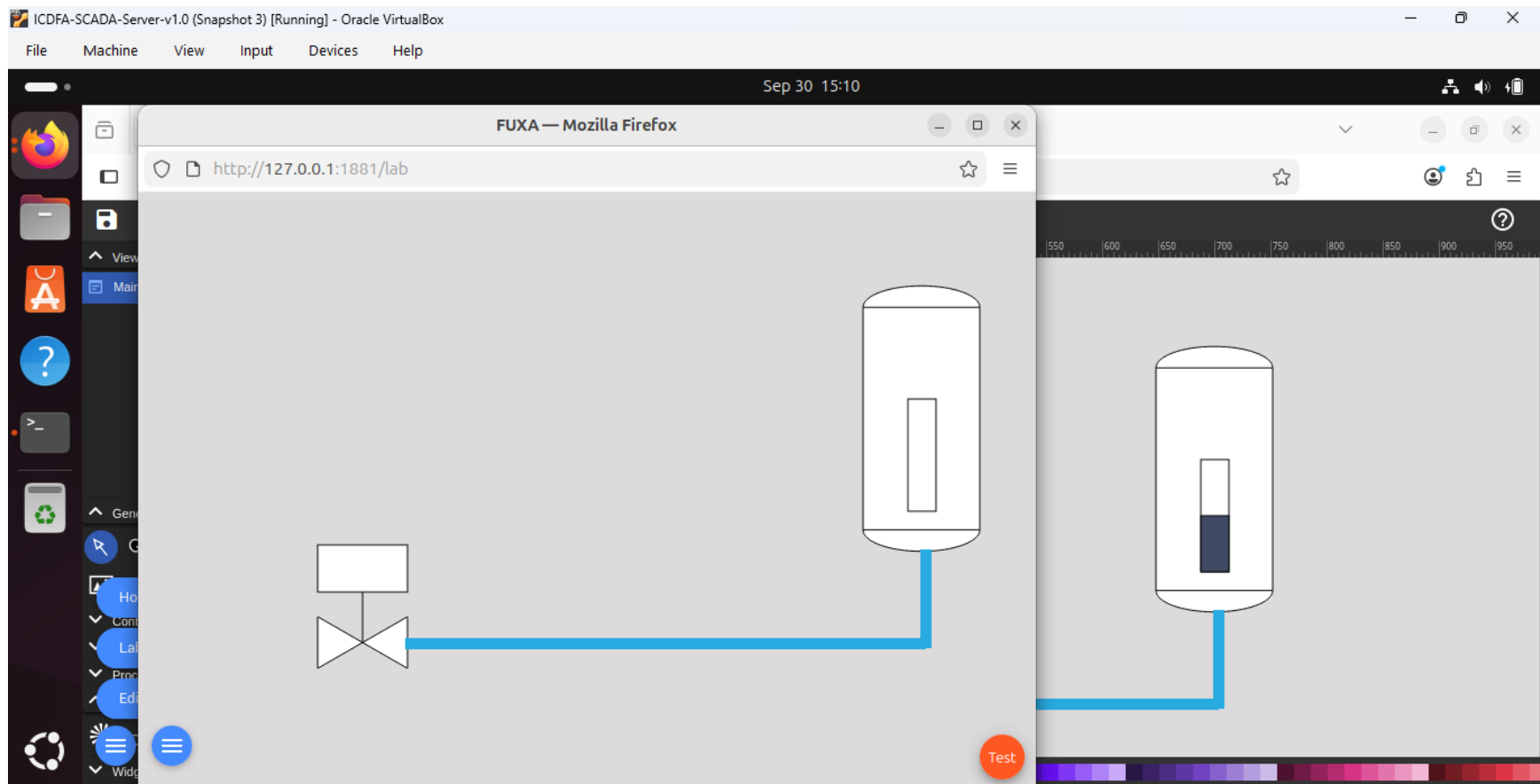


Technical Caption: Confirms active Modbus TCP client communication established between FUXA HMI (192.168.2.10) and OpenPLC (192.168.1.10:502) indicated by the green status highlight.

4. Switched to HMI runtime view (Play mode) and actuated controls to observe visual state changes and continuous polling updates.

SCREENSHOT HERE: Figure 4

Label: *Figure 4: Active FUXA Tank-Fill Process HMI Runtime*



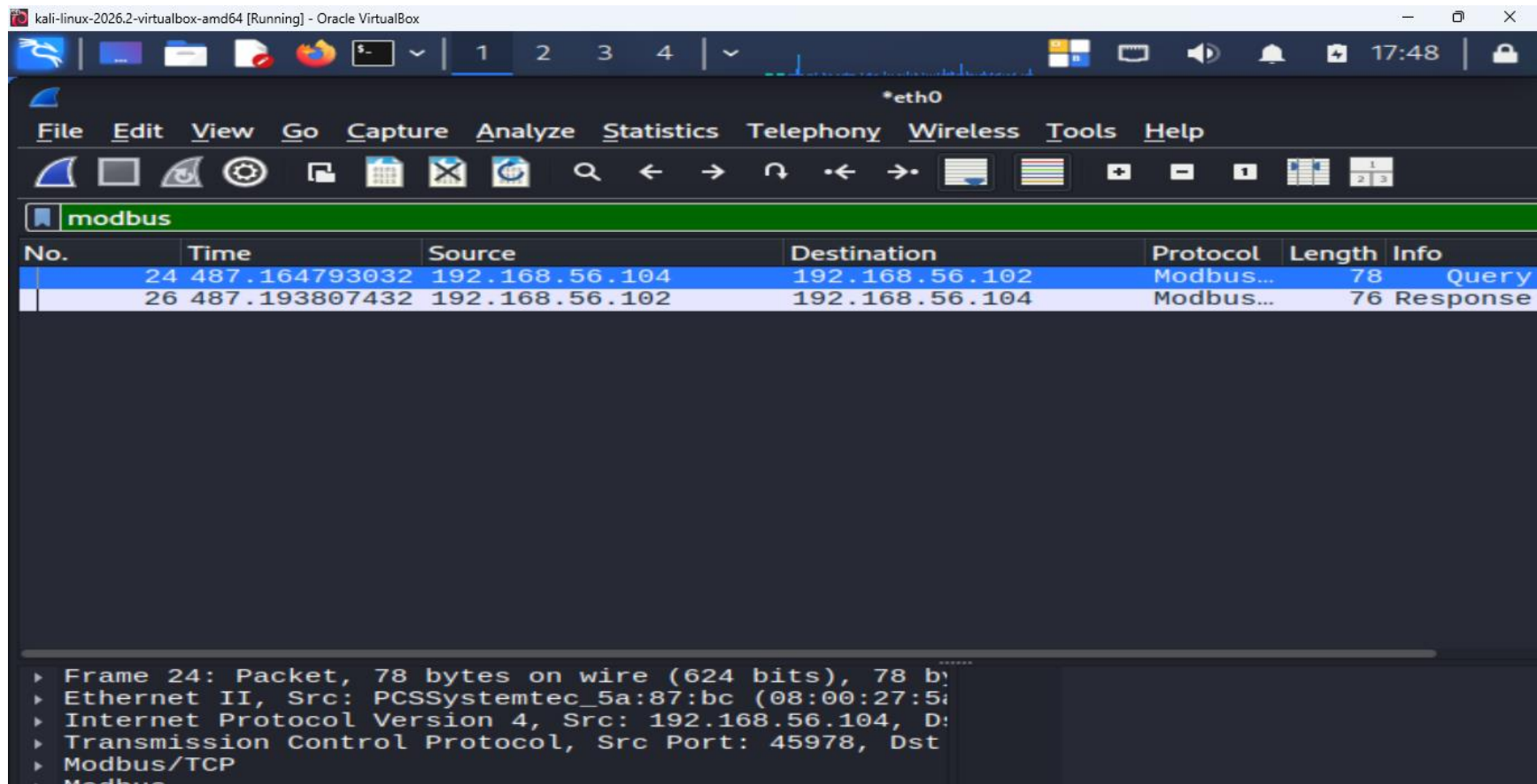
Technical Caption: Demonstrates real-time SCADA monitoring and actuator manipulation over discrete Modbus function codes.

Phase 3: Traffic Capture & Packet Dissection

1. Opened Wireshark on the Kali Linux VM (192.168.2.5), selected interface eth2, and applied the display filter modbus (or tcp.port == 502).
2. Executed a 30-second capture while process updates occurred between the SCADA client and PLC server.

SCREENSHOT HERE: Figure 5

Label: *Figure 5: Wireshark Modbus TCP Traffic Capture Stream*

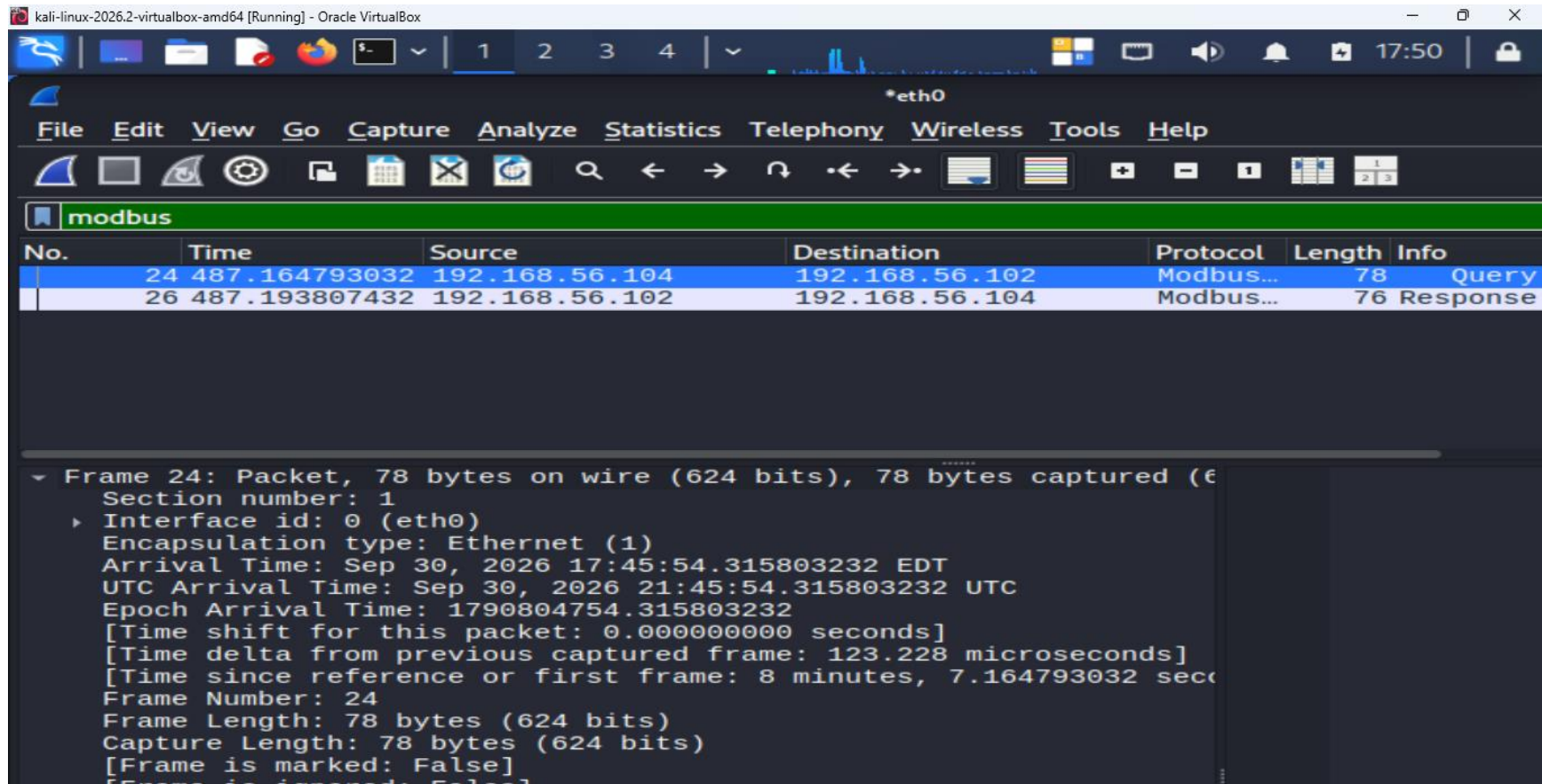


Technical Caption: Captures cleartext Modbus TCP polling requests and responses exchanged between 192.168.2.10 (HMI Client) and 192.168.1.10 (PLC Server).

3. Selected a query/response pair and expanded the Modbus Application Protocol (MBAP) header and Protocol Data Unit (PDU) structures.

SCREENSHOT Figure 6

Label: Figure 6: Expanded Modbus TCP Query and Response Protocol Breakdown

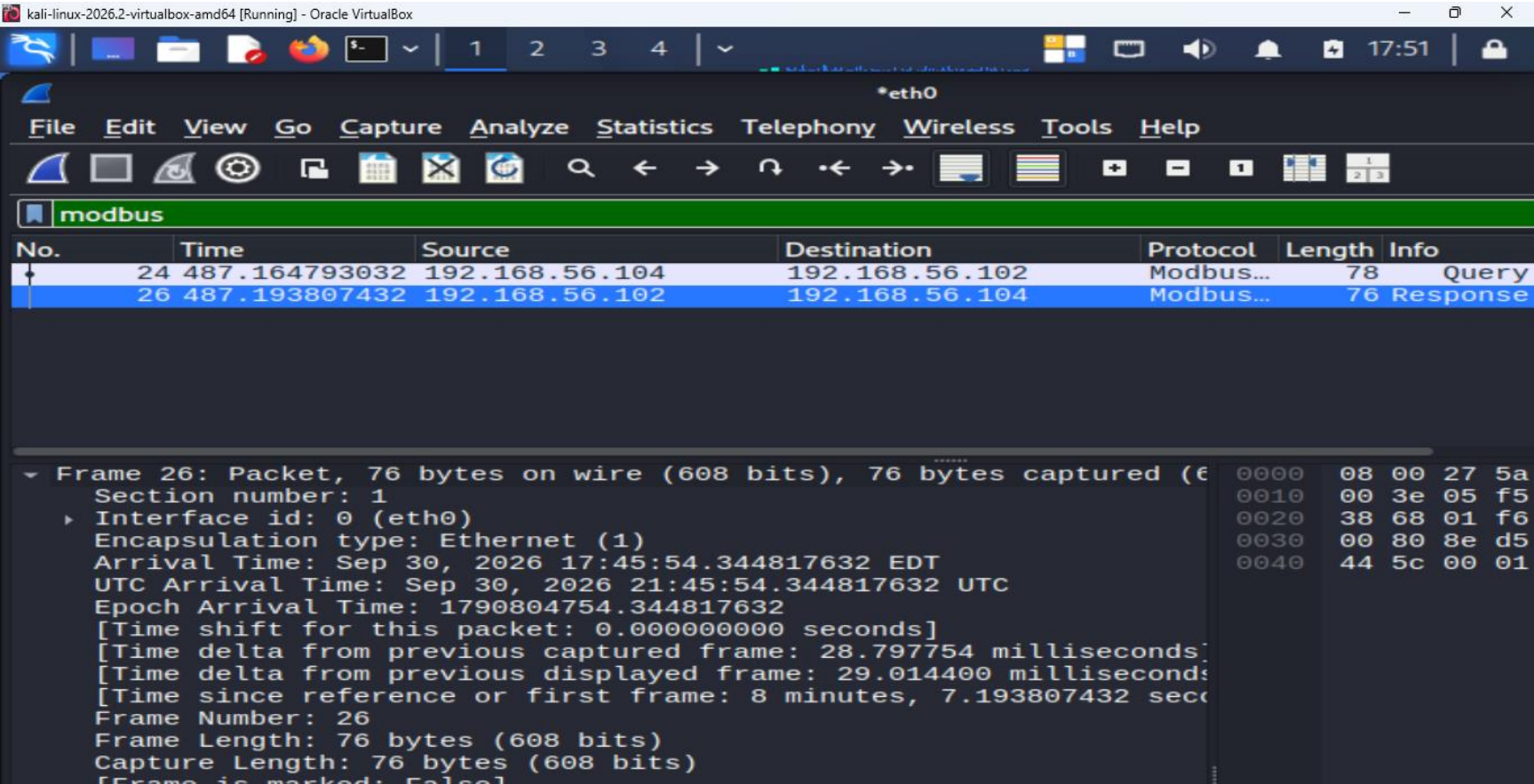


Technical Caption: Dissects the 7-byte MBAP header (Transaction ID, Protocol ID 0x0000, Length, Unit ID 1) and the request PDU.

4. Selected a response frame corresponding to Function Code 01 (Read Coils) to isolate the payload state.

SCREENSHOT : Figure 7

Label: *Figure 7: Modbus FC01 Response Dissection — Discrete State Payload*



Technical Caption: Extracts the bit-level operational binary state (0x01 / TRUE) for output coil variables directly from cleartext industrial payloads.

3. Protocol Reference Tables & Technical Analysis

Modbus Function Codes Reference Table

(Satisfies Step 11 & Rubric Criteria)

Code	Function Name	Bit / Word Size	Data Type & Access	Operational Purpose & OT Context
01	Read Coils	1-bit	Read/Write Discrete Outputs	Reads the state of physical discrete outputs (e.g., FillValve %QX0.0, HighLevel %QX0.3).
02	Read Discrete Inputs	1-bit	Read-Only Discrete Inputs	Reads binary field contact states (e.g., Run %IX0.0, Reset %IX0.1).
03	Read Holding Registers	16-bit word	Read/Write Analog Outputs / Memory	Reads internal 16-bit registers, setpoints, or output parameters.
04	Read Input Registers	16-bit word	Read-Only Analog Inputs	Reads digitized 16-bit analog process measurements (e.g., tank level transmitters, pressure sensors).
05	Write Single Coil	1-bit	Write Single Discrete Output	Forces a single output coil to ON (0xFF00) or OFF (0x0000) (e.g., manually opening a valve).
06	Write Single Holding Register	16-bit word	Write Single Analog Register	Overwrites a single 16-bit numerical value inside a holding register.

Code	Function Name	Bit / Word Size	Data Type & Access	Operational Purpose & OT Context
15	Write Multiple Coils	16-bit packed	Write Sequence of Coils	Overwrites a continuous sequence of discrete output coils in one command.
16	Write Multiple Holding Registers	Multi 16-bit words	Write Sequence of Registers	Writes a continuous block of 16-bit numerical values into consecutive registers.

Dimension / Feature	Modbus TCP/IP	Modbus RTU (Remote Terminal Unit)
OSI Physical Layer	Ethernet (ISO 8802-3), Cat5e/Cat6 Cabling	Serial EIA/TIA-485 (RS-485) or EIA-232 (RS-232)
OSI Transport Layer	TCP/IP over Port 502 (Connection-Oriented)	Direct framing over serial link (No Transport/Network stack)
Architecture	Client / Server Architecture	Primary / Secondary (Master / Slave) Architecture
Framing & Header	7-byte MBAP Header prepended to PDU	1-byte Secondary Address prefix prepended to PDU

Dimension / Feature	Modbus TCP/IP	Modbus RTU (Remote Terminal Unit)
Error Checking	Handled by Ethernet CRC & TCP Checksum	16-bit Cyclic Redundancy Check (CRC-16) trailing frame
Maximum Frame Length	260 Bytes (7-byte MBAP + 253-byte PDU)	256 Bytes (1-byte Addr + 253-byte PDU + 2-byte CRC)

OpenPLC Memory Address Mapping Matrix (Tank_Fill_Sim-WORKING)

Point Name	Variable Type	OpenPLC Location	Modbus Data Area	Modbus Address	Access Type	Function Code
Run	BOOL	%IX0.0	Discrete Inputs	10001 (Input 0)	Read-Only	FC 02
Reset	BOOL	%IX0.1	Discrete Inputs	10002 (Input 1)	Read-Only	FC 02
FillValve	BOOL	%QX0.0	Discrete Output Coils	00001 (Coil 0)	Read / Write	FC 01 / FC 05
DrainValve	BOOL	%QX0.1	Discrete Output Coils	00002 (Coil 1)	Read / Write	FC 01 / FC 05

Point Name	Variable Type	OpenPLC Location	Modbus Data Area	Modbus Address	Access Type	Function Code
CycleComplete	BOOL	%QX0.2	Discrete Output Coils	00003 (Coil 2)	Read / Write	FC 01 / FC 05
HighLevel	BOOL	%QX0.3	Discrete Output Coils	00004 (Coil 3)	Read / Write	FC 01 / FC 05
LowLevel	BOOL	%QX0.4	Discrete Output Coils	00005 (Coil 4)	Read / Write	FC 01 / FC 05

4. Knowledge Check Solutions

Question 1: What transport/networking difference separates Modbus TCP from Modbus RTU?

Response: Modbus TCP operates at Layer 4 (Transport) and Layer 3 (Network) of the OSI model using connection-oriented TCP/IP over Ethernet networks (via TCP port 502). It encapsulates the Modbus Protocol Data Unit (PDU) inside a 7-byte Modbus Application Protocol (MBAP) header and relies on underlying Ethernet hardware and TCP/IP for packet routing and error checking.

In contrast, Modbus RTU operates directly over Layer 1 physical serial media (such as RS-485 or RS-232) without network abstraction. Modbus RTU transmits raw binary serial frames prefixed with a 1-byte slave address and appends a trailing 16-bit Cyclic Redundancy Check (CRC-16) to every frame for bit-level error detection.

Question 2: Why does Modbus use function codes?

Response: Modbus uses function codes to inform the receiving server/PLC what specific operation (Read or Write) is requested and which internal memory table (Coils, Discrete Inputs, Input Registers, or Holding Registers) is being accessed. Function codes standardize command execution across multi-vendor industrial equipment, ensuring controllers parse requests accurately regardless of underlying hardware architecture.

Question 3: What does Function Code 4 request/read?

Response: Function Code 04 (Read Input Registers) requests and reads the contents of 16-bit read-only analog input registers from a server device. In operational technology (OT) systems, Input Registers hold digitized measurements provided by field sensors and transmitters—such as flow meters, temperature sensors, or tank level indicators (e.g., Register 80 / %IW80).

Question 4: Why is visibility of unprotected industrial protocol traffic a cybersecurity concern?

Response: Legacy industrial protocols like standard Modbus TCP transmit all commands, setpoints, addresses, and process measurements in cleartext without native authentication, encryption, or integrity verification.

This lack of security controls presents severe operational risks:

1. **Passive Reconnaissance:** An attacker on the network can use standard packet sniffers (like Wireshark) to inspect cleartext traffic, map out physical process limits, discover PLC memory addresses, and extract system parameters.
2. **Command Injection & MitM Attacks:** Without message authentication, adversaries can forge and inject unauthorized write commands (e.g., FC05 Write Single Coil), manipulate physical actuators (such as opening valves out of sequence), override safety interlocks, or send false telemetry to HMI screens to disrupt industrial operations.